ETHANS CLOUD & AI ACADEMY

Working with

AI APIs in Python

AI APIs in Python

Artificial Intelligence is no longer something that only research laboratories or large technology companies can access. Today, developers and students can send a single Python command and receive a sophisticated AI-generated response within seconds. This is made possible by AI APIs — programming interfaces that give your Python code direct access to large language models developed by companies like OpenAI (the maker of ChatGPT) and Google (the maker of Gemini).

This chapter takes you from zero to a working AI-powered application. You do not need any prior experience with AI or machine learning. Everything is built step by step, starting with setting up your computer correctly, through to writing a complete mini-project that uses a real AI model to do useful work.

What You Will Learn in This Chapter
1. Setting Up the Environment — Python, pip, virtual environments, and API keys
2. Working with the ChatGPT API (OpenAI) — sending requests and reading responses
3. Gemini API Integration (Google AI) — setup and simple usage
4. Prompt Engineering Basics — writing instructions that get better AI outputs
5. Mini Project — building a Text Summariser and Email Generator
6. Error Handling & API Best Practices — rate limits, cost control, and safe coding

API	Application Programming Interface — a set of rules that allows one piece of software to communicate with another. An AI API lets your Python code send text to an AI model and receive its response, without you needing to know how the model works internally.

Section 1

1 Setting Up the Environment

Before writing a single line of AI code, your computer needs to have the right tools installed and configured. This section walks through every step required to create a clean, professional Python development environment — the same setup used by professional developers in the industry.

We will install Python, learn how to manage packages with pip, create an isolated virtual environment for our project, and safely store the API keys required to authenticate with AI services.

1.1 Installing Python

Python is a free, open-source programming language. If you already have Python installed, you can skip ahead to Section 1.2. To check whether Python is available on your computer, open a terminal and run the following command.

```
python --version
# or on some systems:
python3 --version
```

Output:

```
Python 3.11.4
```

If you see a version number starting with 3.8 or higher, you are ready to proceed. If Python is not installed, follow these steps:

1.
2.
3.
4.

📝 Note — Which Python Version?

This chapter uses Python 3.9 or higher. Both the OpenAI and Google Generative AI libraries require Python 3.8 at minimum. Python 2 is no longer supported and will not work with the libraries used here.

If you are on a Mac or Linux system, the command python may refer to Python 2. Use python3 explicitly to ensure you are running the correct version.

1.2 Understanding pip — Python's Package Manager

pip stands for 'Pip Installs Packages'. It is the official tool for installing external Python libraries from the Python Package Index (PyPI) — a public repository containing hundreds of thousands of free packages, including the OpenAI and Google AI libraries we will use in this chapter.

pip	Python's standard package manager. It downloads and installs third-party libraries from the Python Package Index (PyPI) with a single terminal command.

To verify that pip is available on your system, run:

```
pip --version
```

Output:

```
pip 23.2.1 from /usr/local/lib/python3.11/site-packages/pip (python 3.11)
```


The key pip commands you will use throughout this chapter are:

Command	What It Does
pip install package-name	Download and install a package
pip install package-name==2.1.0	Install a specific version of a package
pip uninstall package-name	Remove an installed package
pip list	Show all currently installed packages
pip freeze > requirements.txt	Save all installed packages to a file
pip install -r requirements.txt	Install all packages listed in a file

Table 1.1 — Essential pip commands

1.3 Creating a Virtual Environment

A virtual environment is an isolated Python environment that contains its own set of installed packages, completely separate from your system-wide Python installation. Every professional Python project uses a virtual environment, and for good reason.

🔎 Why Use a Virtual Environment?

Imagine you are working on two projects simultaneously. Project A requires version 1.2 of a library. Project B requires version 2.0 of the same library — and the two versions are not compatible with each other.
Without virtual environments, you can only have one version installed at a time, which means one of your projects will break. With virtual environments, each project gets its own isolated collection of packages, and they never interfere with each other.
Think of it like having separate toolboxes for different jobs — one for woodworking, one for plumbing — rather than throwing all tools into one pile.

Step 1: Create the Virtual Environment

Navigate to the folder where you want to create your project, then run the following command. Replace ai_project with whatever you want to name your project folder.

```
# Navigate to your desired folder first
cd Documents

# Create a new folder for the project
mkdir ai_project
cd ai_project

# Create the virtual environment inside the folder
python -m venv venv
```

This creates a folder named venv inside your project directory. This folder contains a private copy of Python and pip, completely isolated from the rest of your system.

Step 2: Activate the Virtual Environment

After creating the virtual environment, you must activate it. Activation tells your terminal to use the private Python and pip from inside the venv folder rather than the system-wide ones.

Operating System	Activation Command
Windows (Command Prompt)	venv\Scripts\activate
Windows (PowerShell)	venv\Scripts\Activate.ps1
macOS / Linux	source venv/bin/activate

Table 1.2 — Virtual environment activation commands by operating system

Once activated, your terminal prompt changes to show the environment name in parentheses:

```
# Before activation:
C:\Users\Student\Documents\ai_project>

# After activation (Windows):
(venv) C:\Users\Student\Documents\ai_project>

# After activation (Mac/Linux):
(venv) student@computer:~/Documents/ai_project$
```

> 📝 **Note — Always Activate First**
>
> Every time you open a new terminal window and want to work on your project, you must re-activate the virtual environment. The activation only lasts for the current terminal session.
>
> A common beginner mistake is installing packages without the virtual environment being active, which installs them globally instead of into the project. Always look for the (venv) prefix in your terminal prompt before running pip install.

Step 3: Install Required Libraries

With the virtual environment active, install the two main libraries we will use in this chapter:

```
# Install the official OpenAI Python library
pip install openai

# Install Google's Generative AI library
pip install google-generativeai

# Install python-dotenv for managing API keys securely
pip install python-dotenv
```

Output:

```
Successfully installed openai-1.35.7 ...
Successfully installed google-generativeai-0.7.2 ...
Successfully installed python-dotenv-1.0.1 ...
```

1.4 Understanding and Managing API Keys

An API key is a unique secret code that identifies your account when you make requests to an AI service. It is similar to a password — it tells the AI provider who is making the request and allows them to track your usage and charge your account accordingly.

API Key	A unique alphanumeric credential issued by an AI provider that authenticates your application and grants access to their models. Keep it private — anyone with your key can use your account and incur charges on your behalf.

Getting an OpenAI API Key

5.
6.
7.
8.

Getting a Google Gemini API Key

9.
10. **Sign in with a Google account.**
11.
12.
13.

Storing API Keys Safely with .env Files

Never paste your API key directly into a Python file. If you upload that file to GitHub or share it with someone, your key is exposed and your account can be misused. The professional approach is to store keys in a separate .env file and load them using the python-dotenv library.

Step 1: Create a file called .env in your project root folder (not inside any subfolder):

```
# .env file — never share or upload this file
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
GEMINI_API_KEY=AIzaSyxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Step 2: Create a file called .gitignore in the same folder and add the following line to prevent the .env file from being uploaded:

```
.env
```

Step 3: Load the keys in your Python code:

```
from dotenv import load_dotenv
```

```
import os

# Load all variables from the .env file into the environment
load_dotenv()

# Read the key from the environment
openai_key = os.getenv('OPENAI_API_KEY')
gemini_key  = os.getenv('GEMINI_API_KEY')

print('OpenAI key loaded:', openai_key[:8] + '...')  # Print only first 8 chars
```

Output:

```
OpenAI key loaded: sk-xxxxx...
```

⚠️ Critical Security Rule

NEVER hardcode your API key directly in your Python script like this:

```
client = OpenAI(api_key='sk-xxxxxxxxxxxx')   # WRONG — Do not do this
```

If this code file is shared, emailed, or uploaded to a version control system like Git, your API key becomes public. Anyone who finds it can use your account, and you will be billed for their usage. Always load keys from environment variables.

Your final project folder structure should look like this:

```
ai_project/
├── venv/              ← virtual environment (never edit this)
├── .env               ← your secret API keys (never share this)
├── .gitignore         ← tells Git to ignore the .env file
├── main.py            ← your main Python script
└── requirements.txt   ← list of installed packages
```

Section 2

2 Working with the ChatGPT API (OpenAI)

OpenAI is the company behind ChatGPT — one of the most widely known AI systems in the world. Their API allows you to use the same underlying technology in your own Python programs. Instead of typing into the ChatGPT website, your code sends a message programmatically and reads the response — all within seconds.

In this section, you will learn the exact structure of an OpenAI API request, understand the roles involved in a conversation, and write progressively more complex examples.

GPT Model	GPT stands for Generative Pre-trained Transformer. It is the type of AI model that powers ChatGPT. When you call the OpenAI API, you are sending your text to one of these GPT models and receiving its generated response.

2.1 How the OpenAI API Works

The OpenAI API uses a chat-based structure. Every interaction is framed as a conversation between participants called roles. Understanding these roles is the first essential concept.

Role	Who It Represents	Purpose
system	The AI's instructions / persona	Sets the overall behaviour, tone, and rules for the AI in this session
user	The person using the application	Contains the actual question or task being asked
assistant	The AI's previous responses	Used when building multi-turn conversations to give the AI context of what it said before

Table 2.1 — The three message roles in OpenAI's chat format

2.2 Your First OpenAI API Request

Let us write the simplest possible OpenAI API call. This program asks the AI to explain what Python is, in plain English for a beginner.

```
from openai import OpenAI
from dotenv import load_dotenv
import os
```

```
# Step 1: Load the API key from the .env file
load_dotenv()

# Step 2: Create the OpenAI client
client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

# Step 3: Send a message and receive a response
response = client.chat.completions.create(
    model='gpt-4o-mini',          # The AI model to use
    messages=[
        {
            'role': 'system',
            'content': 'You are a helpful teacher who explains technical topics simply.'
        },
        {
            'role': 'user',
            'content': 'What is Python programming in one short paragraph?'
        }
    ]
)

# Step 4: Extract and print the AI's reply
reply = response.choices[0].message.content
print(reply)
```

Output:

Python is a beginner-friendly programming language known for its clean, readable syntax that resembles plain English. It is widely used in fields like data science, web development, and artificial intelligence. Because Python handles many complex technical details automatically, developers can focus on solving problems rather than managing code complexity.

🔎 Breaking Down the Code

client.chat.completions.create() — this is the main method that sends your request to OpenAI.

model='gpt-4o-mini' — specifies which AI model processes your request. gpt-4o-mini is fast and cost-effective; gpt-4o is more capable but costs more per request.

messages=[] — a list of message objects, each with a 'role' and 'content'. This is how you frame the conversation.

response.choices[0].message.content — navigates the response object to extract the AI's actual text reply.

2.3 Understanding the Response Object

The API does not simply return a string. It returns a structured response object containing additional information about the request. Understanding this structure helps you extract exactly what you need and track costs.

```
# Print the full response object to see its structure
print(response)
```

Output:

```
ChatCompletion(
 id='chatcmpl-9XkR...',
 choices=[
  Choice(
   finish_reason='stop',
   index=0,
   message=ChatCompletionMessage(
    content='Python is a beginner-friendly...',
    role='assistant'
   )
  )
 ],
 model='gpt-4o-mini',
 usage=CompletionUsage(
  completion_tokens=68,
  prompt_tokens=41,
  total_tokens=109
 )
)
```

Field	How to Access It	What It Contains
The AI's reply text	response.choices[0].message.content	The actual text generated by the model
Finish reason	response.choices[0].finish_reason	'stop' = completed normally; 'length' = cut off at max_tokens
Tokens used	response.usage.total_tokens	Total input + output tokens consumed (used for billing)

Field	How to Access It	What It Contains
Model used	response.model	Confirms which model processed the request

Table 2.2 — Key fields in the OpenAI response object

2.4 Controlling the Output — Key Parameters

You can customise the AI's behaviour by passing additional parameters to your request. The most important ones are shown below.

```
response = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[
        { 'role': 'user', 'content': 'Write three taglines for a coffee brand.' }
    ],
    temperature=0.9,      # Creativity: 0.0 = very predictable, 1.0 = very creative
    max_tokens=200,       # Maximum length of the AI's response (in tokens)
    n=1,                  # Number of responses to generate
)

print(response.choices[0].message.content)
```

Output:

1. 'Wake Up to What Matters Most.'
2. 'Every Sip, a New Beginning.'
3. 'Crafted for the Moments That Count.'

Parameter	Type	Range	Effect
temperature	float	0.0 – 2.0	Controls creativity. Low = consistent and factual. High = varied and creative.
max_tokens	int	1 – 4096+	Caps the length of the response. 1 token ≈ 0.75 English words.
n	int	1 – 10	Number of independent responses to generate for the same prompt.
top_p	float	0.0 – 1.0	Alternative to temperature. Controls vocabulary diversity.

Table 2.3 — Key parameters for controlling OpenAI API output

2.5 Multi-Turn Conversations

One of the most powerful features of the chat format is the ability to hold multi-turn conversations — where the AI remembers what was said earlier. To achieve this, you must pass the full conversation history in every request. The AI has no memory of previous calls on its own; you are responsible for building and maintaining the history.

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

# Start with a system message and an empty conversation history
conversation = [
    { 'role': 'system', 'content': 'You are a helpful financial advisor.' }
]

def chat(user_message):
    # Add the user's message to the conversation history
    conversation.append({ 'role': 'user', 'content': user_message })

    # Send the full conversation to the API
    response = client.chat.completions.create(
        model='gpt-4o-mini',
        messages=conversation
    )

    # Extract the reply and add it to the history
    reply = response.choices[0].message.content
    conversation.append({ 'role': 'assistant', 'content': reply })

    return reply

# Turn 1
print(chat('What is a mutual fund?'))

# Turn 2 — the AI remembers the previous topic
print(chat('What are the risks involved?'))
```

Output:

Turn 1:

A mutual fund is a pooled investment vehicle where multiple investors
combine their money, which is then managed by a professional fund manager
who invests it in a diversified portfolio of stocks, bonds, or other assets.

Turn 2 (AI understands 'risks' refers to mutual funds from Turn 1):
The main risks include market risk, liquidity risk, and management risk.
Since the fund's value depends on market performance, it can decline...

Section 3

3 Gemini API Integration (Google AI)

Gemini is Google's family of large language models, available through Google AI Studio and the Google Generative AI Python library. While it serves a similar purpose to the OpenAI API — sending text and receiving AI-generated responses — its library structure and terminology differ slightly. Knowing both APIs makes you versatile as a developer.

In this section, you will configure the Gemini library, send your first request, and understand how its response structure compares to OpenAI's.

Feature	OpenAI (GPT) vs Google (Gemini)
Library name	openai vs google-generativeai
Authentication	client = OpenAI(api_key=...) vs genai.configure(api_key=...)
Model access	client.chat.completions.create(model=...) vs genai.GenerativeModel(...)
Sending a message	.create(messages=[...]) vs .generate_content('...')
Reading reply	response.choices[0].message.content vs response.text

Table 3.1 — Side-by-side comparison of OpenAI and Gemini API structures

3.1 Configuring the Gemini Library

Unlike OpenAI, the Gemini library uses a global configuration approach — you configure your API key once at the top of your script and the library handles authentication automatically for all subsequent calls.

```
import google.generativeai as genai
from dotenv import load_dotenv
import os

# Load the API key from the .env file
load_dotenv()

# Configure the library globally — this only needs to be done once
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))

# Verify the setup by listing available models
for model in genai.list_models():
```

```
    if 'generateContent' in model.supported_generation_methods:
        print(model.name)
```

Output:

```
models/gemini-1.5-flash
models/gemini-1.5-pro
models/gemini-1.0-pro
```

📝 Note — Choosing a Gemini Model

gemini-1.5-flash — Fast and cost-effective. Best for most everyday tasks like summarisation, Q&A, and simple generation.

gemini-1.5-pro — More capable, handles longer and more complex documents. Costs more per request.

For all examples in this chapter, we use gemini-1.5-flash to keep costs low while learning.

3.2 Your First Gemini Request

Once configured, sending a request with Gemini requires just two lines of code: creating a model instance and calling generate_content() with your prompt.

```
import google.generativeai as genai
from dotenv import load_dotenv
import os

load_dotenv()
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))

# Step 1: Create a model instance
model = genai.GenerativeModel('gemini-1.5-flash')

# Step 2: Send a prompt and get a response
response = model.generate_content(
    'Explain what an API is in two sentences, using a restaurant analogy.'
)

# Step 3: Access the response text
print(response.text)
```

Output:

Think of an API like a waiter in a restaurant: you (the client application) give the waiter (the API) your order (the request), and the waiter brings back exactly what you asked for from the kitchen (the server) without you needing to enter the kitchen yourself.

Figure 3.1 — A minimal Gemini API request using generate_content()

3.3 Controlling Gemini Output — Generation Config

Gemini allows you to control output behaviour through a GenerationConfig object. This is equivalent to the temperature and max_tokens parameters in the OpenAI API.

```
import google.generativeai as genai
from dotenv import load_dotenv
import os

load_dotenv()
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))

# Define generation configuration
generation_config = genai.GenerationConfig(
    temperature=0.7,         # Creativity level (0.0 to 1.0)
    max_output_tokens=300,    # Maximum response length
    top_p=0.9,               # Vocabulary diversity
)

# Create model with the configuration
model = genai.GenerativeModel(
    model_name='gemini-1.5-flash',
    generation_config=generation_config
)

response = model.generate_content('List 5 benefits of learning to code.')
print(response.text)
```

Output:

1. Problem-solving skills — coding trains logical, structured thinking.
2. Career opportunities — software roles are among the highest-paid globally.
3. Automation — automate repetitive tasks to save hours of manual work.
4. Creativity — build products, tools, and applications from scratch.
5. Understanding technology — helps you make better decisions in a digital world.

3.4 Multi-Turn Chat with Gemini

Gemini provides a built-in chat session object that automatically manages conversation history for you. Unlike OpenAI where you manually maintain the messages list, Gemini's ChatSession stores history internally.

```
import google.generativeai as genai
from dotenv import load_dotenv
import os

load_dotenv()
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))

model = genai.GenerativeModel('gemini-1.5-flash')

# Start a chat session — history is managed automatically
chat = model.start_chat(history=[])

# Turn 1
response1 = chat.send_message('What is cloud computing?')
print('Turn 1:', response1.text)

# Turn 2 — Gemini remembers the context automatically
response2 = chat.send_message('What are the three main types?')
print('Turn 2:', response2.text)

# View the full stored history
print('\n--- Conversation History ---')
for msg in chat.history:
    print(f'{msg.role.upper()}: {msg.parts[0].text[:80]}...')
```

Output:

```
Turn 1: Cloud computing is the delivery of computing services — including servers,
storage, databases, and software — over the internet ('the cloud'), enabling
organisations to scale resources on demand without owning physical hardware.

Turn 2: The three main types of cloud computing are:
1. IaaS (Infrastructure as a Service) — e.g., Amazon EC2
2. PaaS (Platform as a Service) — e.g., Google App Engine
3. SaaS (Software as a Service) — e.g., Gmail, Salesforce

--- Conversation History ---
USER: What is cloud computing?...
MODEL: Cloud computing is the delivery...
USER: What are the three main types?...
MODEL: The three main types...
```

🔎 OpenAI vs Gemini — Chat History Management
OpenAI: You manually append each message to a list and pass the full list every time. You have complete control over what goes into the history.
Gemini: The ChatSession object maintains history internally through chat.history. You just call chat.send_message() and the library handles the rest.
Both approaches achieve the same result. OpenAI gives more control; Gemini is more convenient for simple applications.

Section 4

4 Prompt Engineering Basics

A prompt is the instruction or question you send to an AI model. The quality of the AI's response depends almost entirely on the quality of your prompt. Two people sending requests to the exact same AI model will get dramatically different results if one writes a clear, structured prompt and the other writes a vague one.

Prompt engineering is the practice of designing prompts that reliably produce accurate, useful, and appropriately formatted outputs. It is one of the most practical and immediately valuable skills for anyone working with AI APIs.

Prompt Engineering	The discipline of designing and refining the instructions (prompts) sent to an AI model to maximise the quality, accuracy, and usefulness of the generated responses.

4.1 The Anatomy of an Effective Prompt

A well-crafted prompt typically contains some or all of the following components. You do not always need all of them, but understanding each one gives you tools to improve outputs when the AI is not responding as expected.

Component	What It Does	Example
Role / Persona	Tells the AI who it should act as	'You are an experienced financial analyst.'
Task	Clearly states what the AI must do	'Summarise the following quarterly report.'
Context	Provides background information	'The report covers Q3 2024 for a retail company.'
Format	Specifies the desired output structure	'Respond in bullet points, maximum 5 points.'
Constraints	Sets limits or rules	'Use simple language. Do not include jargon.'
Examples	Shows the AI what good output looks like	'For example: Revenue grew 12% year-on-year.'

Table 4.1 — The six components of a well-structured prompt

4.2 Technique 1 — Be Specific and Detailed

Vague prompts produce vague outputs. The more precisely you describe what you want, the more reliably the AI delivers it. Compare these two prompts for the same task:

Vague Prompt (Weak)	Specific Prompt (Strong)
Write an email about the meeting.	Write a professional email to a client named Priya Mehta confirming a project kickoff meeting on Monday, 21 April at 2:00 PM IST via Google Meet. Mention that the agenda will cover project scope, timelines, and team introductions. Keep the tone formal and the email under 150 words.

Table 4.2 — Vague vs specific prompt comparison for the same task

```
response = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[{
        'role': 'user',
        'content': '''
            Write a professional email to a client named Priya Mehta
            confirming a project kickoff meeting on Monday, 21 April at
            2:00 PM IST via Google Meet.
            Agenda: project scope, timelines, and team introductions.
            Tone: formal. Length: under 150 words.
        '''
    }]
)
print(response.choices[0].message.content)
```

Output:

```
Subject: Project Kickoff Meeting Confirmation — 21 April, 2:00 PM IST

Dear Ms. Mehta,

I hope this message finds you well. I am writing to confirm our project
kickoff meeting scheduled for Monday, 21 April 2025 at 2:00 PM IST,
to be held via Google Meet.

The agenda for the session will cover the following:
  - Project scope and objectives
  - Timelines and key milestones
  - Team introductions

Please do not hesitate to contact me should you require any changes.
```

```
Warm regards,
[Your Name]
```

4.3 Technique 2 — Assign a Role to the AI

The system message is the most powerful tool for shaping how the AI behaves throughout a conversation. Use it to assign a persona, set a tone, or restrict the scope of the AI's responses.

```
# Without a system role — generic response
response = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[{ 'role': 'user', 'content': 'Explain recursion.' }]
)
print('Generic:', response.choices[0].message.content[:120])

# With a targeted system role — focused, audience-appropriate response
response = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[
      {
        'role': 'system',
        'content': '''You are a computer science professor teaching
                      first-year undergraduate students. Explain concepts
                      using relatable analogies. Avoid jargon.'''
      },
      { 'role': 'user', 'content': 'Explain recursion.' }
    ]
)
print('Targeted:', response.choices[0].message.content[:200])
```

Output:

```
Generic: Recursion is a programming technique where a function calls itself...

Targeted: Imagine you are standing in a long queue at a cinema and you want
to know how many people are ahead of you. Instead of counting yourself,
you ask the person in front of you: 'How many people are ahead of you?'
They ask the person in front of them, and so on, until someone at the front
says 'zero'. That is recursion — a problem solved by breaking it into the
same, smaller version of itself.
```

4.4 Technique 3 — Specify the Output Format

When your application needs to process the AI's output programmatically — for example, to display it in a table or pass it to another function — always specify the exact format you need. JSON is particularly useful because Python can parse it directly.

```
import json

response = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[{
        'role': 'user',
        'content': '''
            Provide information about three popular Python libraries.
            Respond ONLY with valid JSON in exactly this format — no extra text:
            [
              {
                'name': 'library name',
                'purpose': 'one sentence description',
                'example_use': 'one concrete use case'
              }
            ]
        '''
    }]
)

# Parse the JSON response directly
libraries = json.loads(response.choices[0].message.content)
for lib in libraries:
    print(f"Library: {lib['name']}")
    print(f"Purpose: {lib['purpose']}")
    print(f"Use Case: {lib['example_use']}")
    print()
```

Output:

```
Library: pandas
Purpose: A powerful library for data manipulation and analysis.
Use Case: Cleaning and analysing a CSV file of sales transactions.

Library: requests
Purpose: Simplifies making HTTP requests to web APIs and websites.
Use Case: Fetching live weather data from a public REST API.

Library: matplotlib
Purpose: Creates static, interactive, and animated visualisations.
Use Case: Plotting a monthly revenue trend as a line chart.
```

4.5 Technique 4 — Few-Shot Prompting

Few-shot prompting means giving the AI two or three examples of input-output pairs before presenting the actual task. This teaches the AI the exact pattern and style you want, making responses far more consistent and predictable.

```
response = client.chat.completions.create(
  model='gpt-4o-mini',
  messages=[{
    'role': 'user',
    'content': '''
      Convert the following customer reviews into a structured format.
      Sentiment must be: Positive, Negative, or Neutral.

      Example 1:
      Review: 'The product arrived on time and works perfectly.'
      Output: Sentiment: Positive | Key Issue: None | Action: None

      Example 2:
      Review: 'Packaging was damaged and the item was missing parts.'
      Output: Sentiment: Negative | Key Issue: Damaged packaging, missing parts | Action: Refund or
replacement

      Now process this review:
      Review: 'The quality is decent but delivery took 2 weeks longer than promised.'
      Output:
    '''
  }]
)
print(response.choices[0].message.content)
```

Output:

Sentiment: Negative | Key Issue: Delayed delivery | Action: Investigate logistics partner

💡 Pro Tip — Chain of Thought Prompting

For complex reasoning tasks, add the instruction 'Think step by step' to your prompt. This technique — called Chain of Thought prompting — causes the AI to reason through the problem incrementally rather than jumping to a conclusion, resulting in more accurate answers for maths, logic, and analytical tasks.

Example: 'A company had revenue of 4.2 crore last year and grew by 18% this year. Think step by step to calculate this year's revenue.'

The AI will show its working: 4.2 crore × 0.18 = 0.756 crore increase → 4.2 + 0.756 = 4.956 crore.

Section 5

5 Mini Project — Text Summariser & Email Generator

In this section, we bring together everything covered so far to build two practical, real-world applications. Both projects are designed to be immediately useful in a professional context — the kind of tools that save time and demonstrate the genuine power of integrating AI into your workflow.

Project A builds an intelligent text summariser that condenses long articles or documents into concise bullet-point summaries. Project B builds an email generator that produces professional, context-aware emails from a brief description.

Project Overview
Project A — Smart Text Summariser
Input: A long block of text (article, report, meeting notes, etc.)
Output: A structured summary with key points, main conclusion, and action items
API Used: OpenAI GPT-4o-mini
Project B — Professional Email Generator
Input: A short description of the email's purpose, recipient, and tone
Output: A complete, ready-to-send professional email with subject line
API Used: Google Gemini 1.5 Flash

5.1 Project A — Smart Text Summariser

Project Structure

```
ai_project/
├── venv/
├── .env
├── summariser.py      ← Project A
└── email_generator.py ← Project B
```

```
└── requirements.txt
```

Complete Code — summariser.py

This summariser accepts any block of text and returns a structured summary containing the main topic, five key bullet points, the core conclusion, and any recommended action items.

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

def summarise_text(text: str) -> dict:
    '''
    Summarise a block of text using GPT-4o-mini.
    Returns a dictionary with structured summary fields.
    '''
    prompt = f'''
    You are a professional document analyst. Summarise the following text.
    Respond ONLY with valid JSON in exactly this structure:
    {{
      'main_topic':    'one sentence describing the primary subject',
      'key_points':    ['point 1', 'point 2', 'point 3', 'point 4', 'point 5'],
      'conclusion':    'one sentence capturing the core takeaway',
      'action_items':  ['action 1', 'action 2']
    }}

    TEXT TO SUMMARISE:
    {text}
    '''

    response = client.chat.completions.create(
        model='gpt-4o-mini',
        messages=[{ 'role': 'user', 'content': prompt }],
        temperature=0.3   # Low temperature for consistent, factual summaries
    )

    import json
    return json.loads(response.choices[0].message.content)


def print_summary(summary: dict):
```

```
    '''Display the structured summary in a readable format.'''
    print('=' * 60)
    print('DOCUMENT SUMMARY')
    print('=' * 60)
    print(f"Main Topic: {summary['main_topic']}")
    print()
    print('Key Points:')
    for i, point in enumerate(summary['key_points'], 1):
        print(f'  {i}. {point}')
    print()
    print(f"Conclusion: {summary['conclusion']}")
    print()
    print('Recommended Actions:')
    for action in summary['action_items']:
        print(f'  - {action}')
    print('=' * 60)


# ── Test the summariser ──────────────────────────────────────────────
sample_text = '''
India's digital payments ecosystem has undergone a remarkable transformation
over the past decade. The Unified Payments Interface (UPI), launched by the
National Payments Corporation of India in 2016, processed over 13 billion
transactions in a single month in 2024. Small merchants, street vendors, and
rural households that previously operated exclusively in cash now conduct
daily transactions digitally. This shift has been driven by smartphone
penetration, government incentives, and the simplicity of QR-code-based
payments. However, challenges remain: cybersecurity threats are increasing,
digital literacy gaps persist in certain demographics, and rural internet
infrastructure requires further investment to sustain this growth trajectory.
'''

summary = summarise_text(sample_text)
print_summary(summary)
```

Output:

```
============================================================
DOCUMENT SUMMARY
============================================================
Main Topic: The rapid growth and challenges of India's digital payments
ecosystem, driven primarily by UPI adoption.

Key Points:
  1. UPI processed over 13 billion transactions in a single month in 2024.
  2. Adoption has expanded to small merchants, vendors, and rural households.
  3. Growth drivers include smartphone penetration and QR-code simplicity.
```

```
  4. Cybersecurity threats are a growing concern in the ecosystem.
  5. Digital literacy and rural infrastructure remain key challenges.

Conclusion: India's UPI-driven digital payment revolution is globally
significant but requires continued investment in security and infrastructure.

Recommended Actions:
  - Invest in cybersecurity infrastructure and fraud detection systems.
  - Develop digital literacy programmes for underserved demographics.
============================================================
```

5.2 Project B — Professional Email Generator

Complete Code — email_generator.py

This generator accepts a simple description of an email's purpose and produces a complete, ready-to-send professional email with a subject line and appropriate sign-off.

```
import google.generativeai as genai
from dotenv import load_dotenv
import os

load_dotenv()
genai.configure(api_key=os.getenv('GEMINI_API_KEY'))

model = genai.GenerativeModel(
    model_name='gemini-1.5-flash',
    generation_config=genai.GenerationConfig(temperature=0.6, max_output_tokens=500)
)

def generate_email(
    purpose: str,
    recipient_name: str,
    sender_name: str,
    tone: str = 'formal'
) -> str:
    '''
    Generate a professional email using Gemini.

    Args:
        purpose      : Brief description of what the email should accomplish.
        recipient_name: Full name of the recipient.
        sender_name   : Full name of the sender.
        tone         : 'formal', 'semi-formal', or 'friendly'.
```

```
    Returns:
        A complete email as a formatted string.
    '''
    prompt = f'''
    You are a professional business communication specialist.
    Write a {tone} email based on the following details:

    Recipient Name : {recipient_name}
    Sender Name    : {sender_name}
    Purpose        : {purpose}
    Tone           : {tone}

    Include:
    - A clear, relevant subject line on the first line (format: Subject: ...)
    - A proper salutation
    - A concise body (maximum 3 short paragraphs)
    - A professional closing and sender name
    '''

    response = model.generate_content(prompt)
    return response.text


# ── Test Case 1: Project Follow-Up ──────────────────────────────
email1 = generate_email(
    purpose='Follow up on the website redesign project status.',
    recipient_name='Rohan Iyer',
    sender_name='Kavya Nair',
    tone='semi-formal'
)
print(email1)
print('─' * 60)

# ── Test Case 2: Interview Invitation ───────────────────────────
email2 = generate_email(
    purpose='Invite candidate Arjun Kapoor for a second-round technical interview on Friday, 25 April at 11:00 AM.',
    recipient_name='Arjun Kapoor',
    sender_name='Priya Sharma, HR Manager',
    tone='formal'
)
print(email2)
```

Output:

```
Subject: Following Up on Website Redesign Project

Hi Rohan,
```

I hope you are doing well. I wanted to check in on the progress of the website redesign project. Could you share a brief update on the current status and any blockers the team is facing?

I would appreciate an update by end of week so we can adjust timelines if needed. Please let me know if you need anything from my end.

Thanks,
Kavya Nair

Subject: Invitation for Second-Round Technical Interview

Dear Mr. Kapoor,

I am pleased to inform you that you have been shortlisted for a second-round technical interview for the position you applied for.

The interview is scheduled for Friday, 25 April 2025 at 11:00 AM. Further details regarding the format and interviewers will be shared separately. Please confirm your availability at your earliest convenience.

Warm regards,
Priya Sharma
HR Manager

📝 **Extending the Projects**
Text Summariser extensions:
- Accept a file path as input and read the text from a .txt or .pdf file
- Add a word count limit parameter to control summary length
- Save summaries to a JSON file with timestamps
Email Generator extensions:
- Add a department parameter to automatically include CC recipients
- Support multiple languages by adding a language parameter
- Integrate with smtplib to send the generated email directly

Section 6

6 Error Handling & API Best Practices

When working with external APIs, many things can go wrong that are entirely outside your control — the network might be unreliable, you might exceed the API provider's rate limit, your API key might expire, or the service might be temporarily unavailable. Professional code anticipates these failures and handles them gracefully rather than crashing.

This section also covers best practices for managing API costs and writing production-ready code — skills that separate a student project from a professional application.

6.1 Common API Errors and Their Causes

Error Type	HTTP Code	Common Cause	How to Handle
AuthenticationError	401	Invalid or missing API key	Check .env file; regenerate key if needed
RateLimitError	429	Too many requests in a short time	Add retry logic with exponential backoff
APIConnectionError	—	No internet / server down	Retry after delay; check network status
BadRequestError	400	Invalid request parameters	Validate inputs before sending request
APITimeoutError	408	Request took too long to respond	Increase timeout setting; retry once

Table 6.1 — Common API errors, their causes, and recommended handling strategies

6.2 Implementing Robust Error Handling

The openai library provides specific exception classes for each error type, allowing you to write targeted error handling rather than catching all errors generically.

```
from openai import OpenAI, AuthenticationError, RateLimitError,
                APIConnectionError, APITimeoutError
from dotenv import load_dotenv
import os
import time
```

```
load_dotenv()
client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

def safe_api_call(prompt: str, retries: int = 3) -> str:
    '''
    Make an API call with error handling and automatic retry logic.

    Args:
        prompt  : The user's message to send.
        retries : Number of times to retry on transient errors.

    Returns:
        The AI's response text, or an error message string.
    '''
    for attempt in range(retries):
        try:
            response = client.chat.completions.create(
                model='gpt-4o-mini',
                messages=[{ 'role': 'user', 'content': prompt }],
                timeout=30   # Maximum seconds to wait for a response
            )
            return response.choices[0].message.content

        except AuthenticationError:
            # API key is wrong — retrying will not help
            return 'Error: Invalid API key. Please check your .env file.'

        except RateLimitError:
            if attempt < retries - 1:
                wait_time = 2 ** attempt   # Exponential backoff: 1s, 2s, 4s
                print(f'Rate limit reached. Waiting {wait_time}s before retry...')
                time.sleep(wait_time)
            else:
                return 'Error: Rate limit exceeded. Please wait before trying again.'

        except APIConnectionError:
            if attempt < retries - 1:
                print(f'Connection error. Retrying (attempt {attempt + 2}/{retries})...')
                time.sleep(2)
            else:
                return 'Error: Cannot connect to OpenAI. Check your internet connection.'

        except APITimeoutError:
            return 'Error: The request timed out. Please try again.'
```

```
        except Exception as e:
            # Catch any unexpected errors
            return f'Unexpected error: {str(e)}'

    return 'Error: All retry attempts failed.'


# Test the function
result = safe_api_call('What are the key features of Python 3.12?')
print(result)
```

🔎 What is Exponential Backoff?

When you hit a rate limit, the worst thing you can do is immediately retry — you will just hit the limit again. Exponential backoff is a retry strategy where each retry waits progressively longer:

Attempt 1 failed → wait 1 second → retry

Attempt 2 failed → wait 2 seconds → retry

Attempt 3 failed → wait 4 seconds → retry

This gives the API server time to recover and reduces the chance of being temporarily blocked. The formula is: wait_time = 2 ^ attempt_number.

6.3 Input Validation Before API Calls

Never send a request to the API without first validating the user's input. Empty prompts, excessively long inputs, or inputs with problematic content can result in wasted API calls and unnecessary charges.

```
def validate_and_call(user_input: str) -> str:
    '''Validate input before making an API call.'''

    # Check 1: Input must not be empty
    if not user_input or not user_input.strip():
        return 'Error: Please provide a non-empty prompt.'

    # Check 2: Input must not be too short to be meaningful
```

```
    if len(user_input.strip()) < 10:
        return 'Error: Prompt is too short. Please provide more context.'

    # Check 3: Input must not exceed the token limit
    # Rough estimate: 1 token ≈ 4 characters for English text
    estimated_tokens = len(user_input) / 4
    if estimated_tokens > 3000:
        return 'Error: Input is too long. Please shorten to under 3000 tokens.'

    # All checks passed — proceed with the API call
    return safe_api_call(user_input)


# Test validation
print(validate_and_call(''))          # Error: empty
print(validate_and_call('hi'))        # Error: too short
print(validate_and_call('Explain APIs')) # Valid — proceeds to API call
```

Output:

```
Error: Please provide a non-empty prompt.
Error: Prompt is too short. Please provide more context.
An API (Application Programming Interface) is a set of rules...
```


6.4 Managing API Costs

AI APIs charge based on the number of tokens processed — both in your prompt (input) and in the AI's response (output). Understanding and controlling token usage is essential to keep costs predictable.

Token	The basic unit of text that AI models process. A token is approximately 0.75 English words, or 4 characters. 'Hello, World!' is roughly 4 tokens. A 1,000-word document is roughly 1,333 tokens.

Cost Control Strategy	Implementation	Typical Saving
Use smaller models for simple tasks	Use gpt-4o-mini instead of gpt-4o for classification, Q&A, and summarisation	Up to 95% cost reduction
Set max_tokens limits	max_tokens=200 prevents runaway long responses	Prevents cost spikes
Monitor usage per call	Log response.usage.total_tokens for every API call	Identifies expensive prompts

Cost Control Strategy	Implementation	Typical Saving
Cache repeated responses	Store results for identical prompts; return cached version	Eliminates duplicate charges
Validate input length	Reject inputs over a token limit before calling the API	Prevents oversized requests

Table 6.2 — Practical cost control strategies for production API usage

```
def call_with_cost_tracking(prompt: str) -> dict:
    '''Make an API call and return both the response and its cost.'''

    response = client.chat.completions.create(
        model='gpt-4o-mini',
        messages=[{ 'role': 'user', 'content': prompt }],
        max_tokens=300
    )

    usage = response.usage

    # GPT-4o-mini pricing (approximate as of mid-2024)
    # $0.00015 per 1,000 input tokens
    # $0.00060 per 1,000 output tokens
    input_cost  = (usage.prompt_tokens     / 1000) * 0.00015
    output_cost = (usage.completion_tokens / 1000) * 0.00060
    total_cost  = input_cost + output_cost

    return {
        'reply'          : response.choices[0].message.content,
        'input_tokens'   : usage.prompt_tokens,
        'output_tokens'  : usage.completion_tokens,
        'total_tokens'   : usage.total_tokens,
        'estimated_cost_usd': round(total_cost, 6)
    }


result = call_with_cost_tracking('Summarise the benefits of cloud computing in 3 points.')
print('Response:', result['reply'])
print(f"Tokens used: {result['total_tokens']} (input: {result['input_tokens']}, output: {result['output_tokens']})")
print(f"Estimated cost: ${result['estimated_cost_usd']}")
```

Output:

```
Response: 1. Scalability — instantly scale resources up or down...
Tokens used: 89 (input: 27, output: 62)
Estimated cost: $0.000041
```

6.5 API Best Practices Checklist

Before deploying any application that uses AI APIs, verify that all of the following practices are in place:

Practice	Why It Matters	Status
API keys stored in .env file	Prevents credential exposure in shared code	[ ] Check
.env listed in .gitignore	Prevents accidental upload of credentials to Git	[ ] Check
Input validation before API calls	Prevents empty/oversized requests and wasted cost	[ ] Check
try/except on every API call	Prevents crashes from network or authentication failures	[ ] Check
Retry logic with backoff	Handles transient rate limit errors gracefully	[ ] Check
max_tokens parameter set	Prevents unexpectedly long (and expensive) responses	[ ] Check
Token usage logged per call	Enables cost monitoring and optimisation	[ ] Check
Smallest viable model chosen	Maximises cost-efficiency without sacrificing quality	[ ] Check

Table 6.3 — Production readiness checklist for AI API applications

💡 Pro Tip — Setting Spending Limits

Both OpenAI and Google AI provide spending limit settings in their developer dashboards. Before starting any project, navigate to your account's Billing or Usage settings and set a monthly spending limit.

OpenAI: platform.openai.com ⟶ Billing ⟶ Usage limits ⟶ Set a monthly budget

Google AI: aistudio.google.com ⟶ Account settings ⟶ Billing

Even a small limit of ₹500-1,000 per month will protect you from unexpected charges during learning and development.

Chapter Summary

This chapter covered the complete workflow for building Python applications that communicate with AI APIs. Below is a structured summary of every key concept covered across the six sections.

Section	Topic	Key Concepts
1	Setting Up the Environment	Python install, pip, virtual environments, .env files, API key security
2	OpenAI ChatGPT API	Roles (system/user/assistant), chat.completions.create(), response object, multi-turn conversations
3	Google Gemini API	genai.configure(), GenerativeModel, generate_content(), start_chat(), history management
4	Prompt Engineering	Specificity, role assignment, format specification, few-shot prompting, chain-of-thought
5	Mini Project	Text summariser with JSON output parsing, email generator with Gemini, structured prompts
6	Error Handling & Best Practices	Exception types, exponential backoff, input validation, token cost tracking, spending limits

Table S.1 — Chapter Summary Reference

- A virtual environment isolates your project's packages from the system. Always activate it before installing libraries or running code.
- API keys are credentials — never hardcode them in your Python files. Use .env files and the python-dotenv library.
- The OpenAI API uses a messages list with three roles: system (behaviour), user (input), and assistant (previous AI replies). You manually maintain the conversation history.
- Gemini's ChatSession automatically tracks conversation history. genai.configure() authenticates globally; generate_content() sends a request.
- Prompt quality directly determines output quality. Use specificity, assigned roles, output format instructions, few-shot examples, and chain-of-thought for progressively better results.
- All API calls must be wrapped in try/except blocks. Use exponential backoff for rate limit errors. Validate all user inputs before making requests.
- Token usage determines cost. Use the smallest model appropriate for the task, set max_tokens limits, and monitor usage per call to keep costs predictable.

© AI APIs in Python — Confidential Course Material